



Lab 10 – Worksheet

Name: Sualeh, Sarosh, Hamza, Ghazi	ID:10303	Section:
------------------------------------	----------	----------

Task a. 10-bit counter

Code: Design module `h_counter` & testbench

Provide appropriately commented code for designed module, the code should contain meaningful variable naming. Modify the testbench provided in Fig 10.9(b) to verify the functionality of your designed `h_counter` module.

**Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.*

DESIGN MODULE:-

SIMULATION - Behavioral Simulation - Functional - sim_1 - test_h_counter

Flow Navigator | Scope | Sources | Objects | Protocol Instances

C:/Users/Administrator/Desktop/project_3/project_3/project_3.srcs/sources_1/new/h_counter.v

```
1 module h_counter(  
2     output reg [9:0] count,  
3     output reg trig_v,  
4     input clk  
5 );  
6  
7 initial begin  
8     count = 10'b000_000_000_0;  
9     trig_v = 0;  
10 end  
11  
12 always @(posedge clk) begin  
13     if (count == 10'b1100011111) begin // 799 in binary  
14         trig_v <= 1;  
15         count <= 10'b000_000_000_0; // Reset count  
16     end  
17     else begin  
18         count <= count + 1'b1;  
19         trig_v <= 0;  
20     end  
21 end  
22  
23 endmodule  
24
```

**TEST BENCH:-**

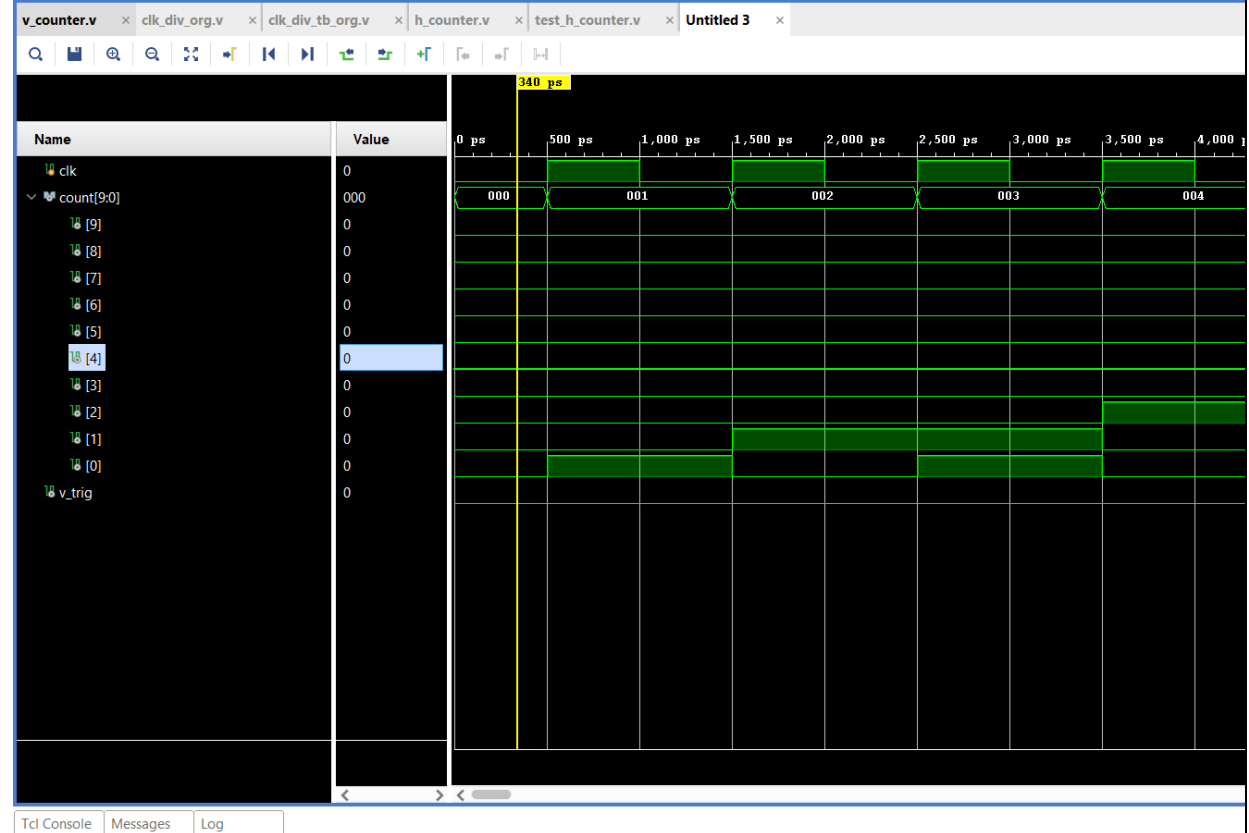
```
TopLevel_tb.v x TopLevelModule.v x v_counter.v x test_v_vounter.v x test_h_counter.v x
C:/Users/Administrator/Desktop/project_3/project_3.srscs/sim_1/new/test_h_counter.v

1 module test_h_counter();
2     reg clk;
3     wire [9:0] count;
4     wire v_trig;
5     // Instantiate the counter module
6     h_counter c1 (.clk(clk), .count(count), .trig_v(v_trig));
7
8     // Initialize clock
9     initial clk = 1'b0;
10
11    // Generate clock signal with a period of 10 time units (T = 10, so toggle every 5 units)
12    always #0.5 clk = ~clk;
13
14    // Stop simulation after 200 time units
15    initial #1000 $finish;
16 endmodule
17
```



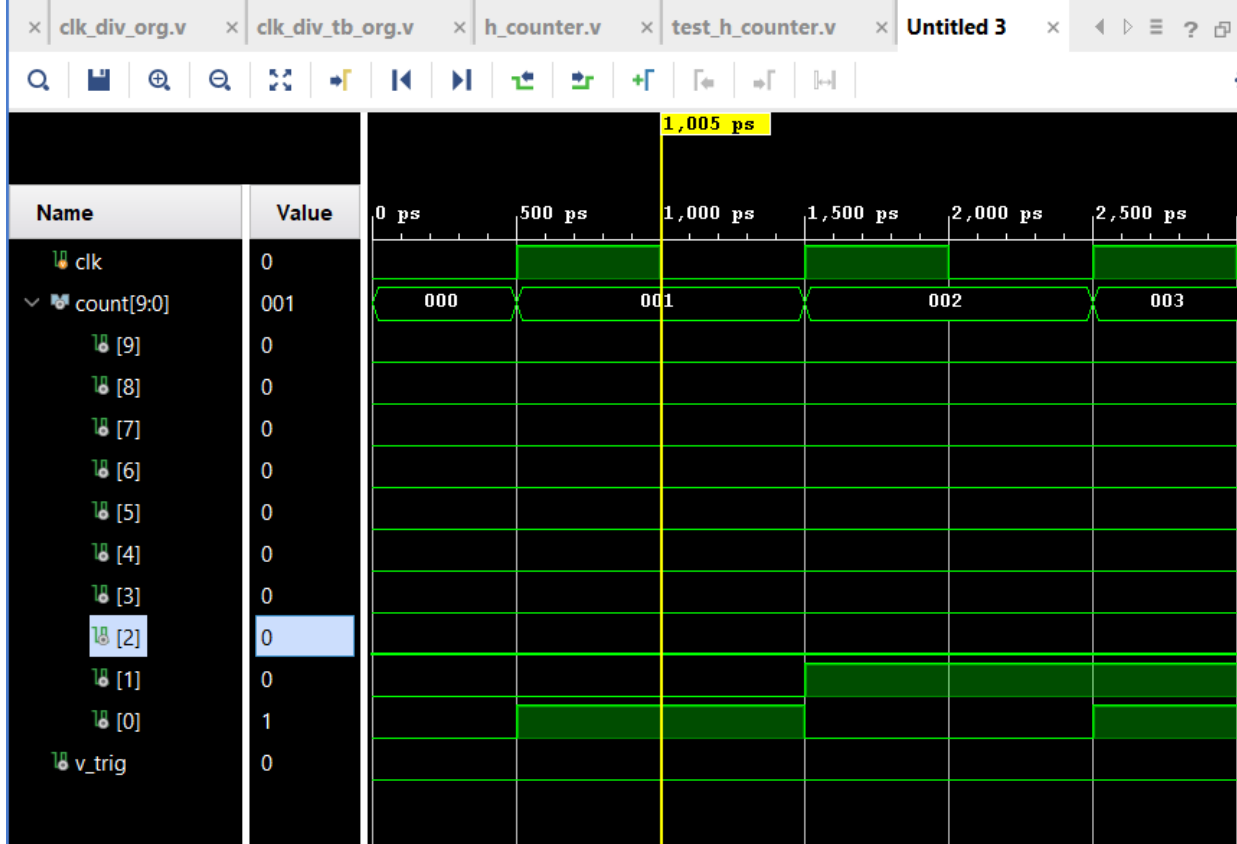
RESULTS:-

CASE 1: AT INSTANT ZERO, THE VALUE OF COUNT=0



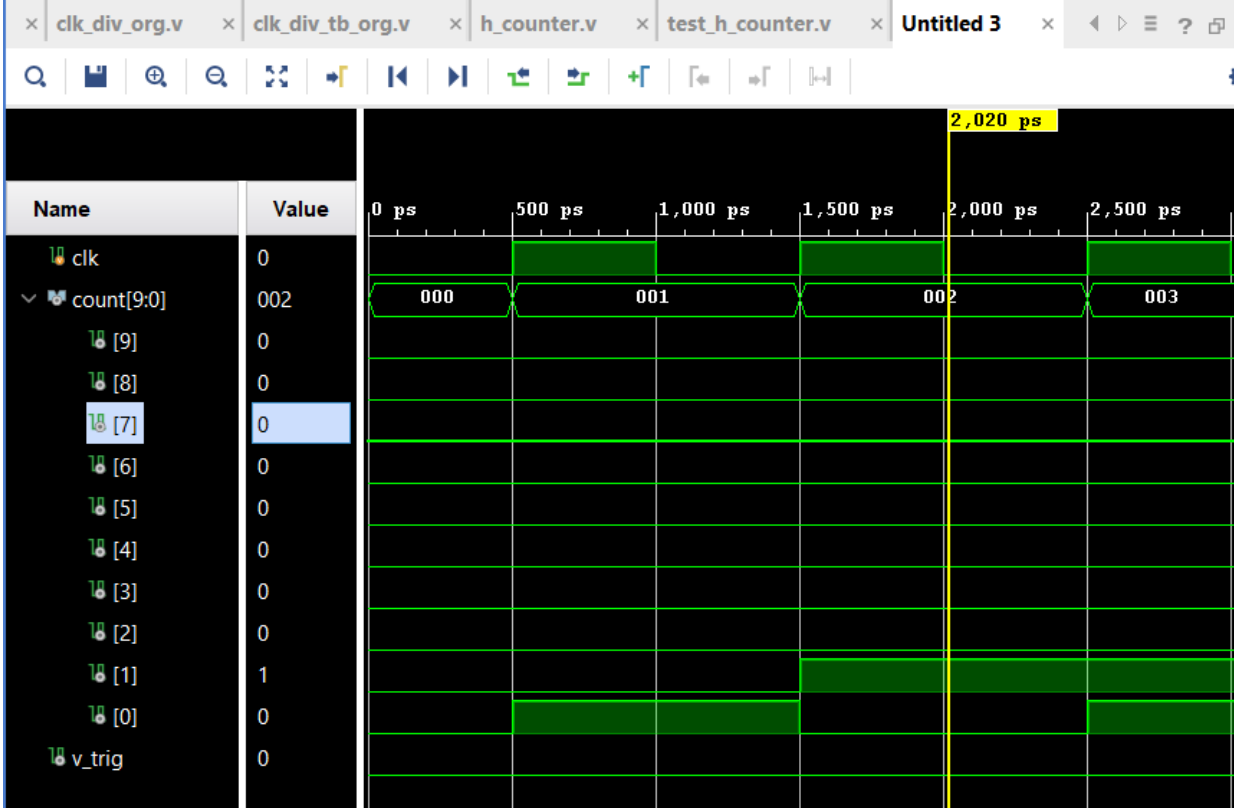


CASE 2: AT INSTANT '1', THE VALUE OF COUNT=1



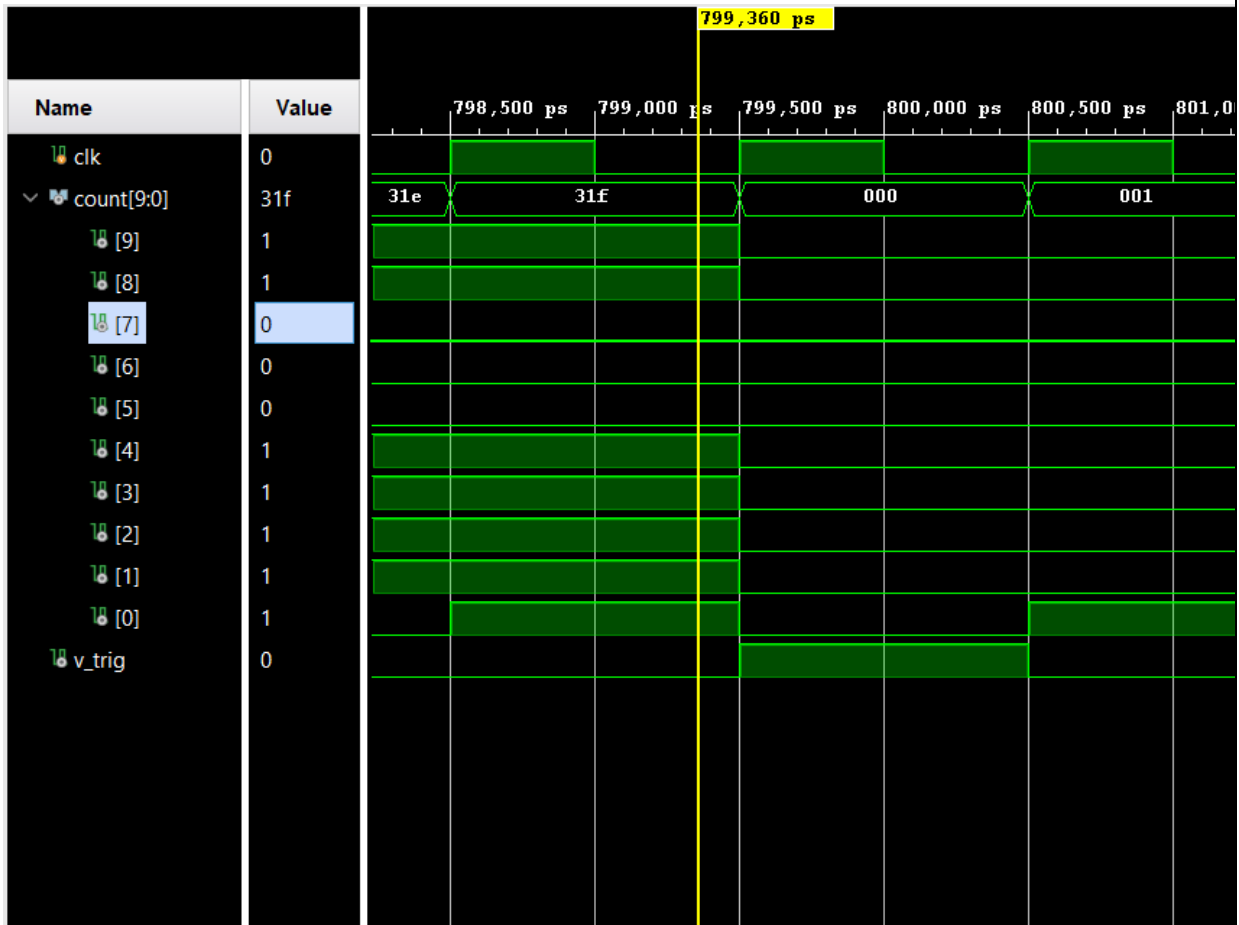


CASE 3: AT INSTANT '2', THE VALUE OF COUNT=2





CASE 4: WHEN THE COUNT REACHES 799 ie. 1111100011. THE V_TRIG TURNS 'HIGH' AND THE CLOCK RESETS.



Comments

**Observation/Comments on the obtained results/working of code.*

- The counter increments on every positive edge.
- The counter (count) is a 10-bit register, meaning it can hold values from 0 to 1023. However, it resets at 799 (binary 1100011111), meaning it only counts from 0 to 799.
- trig_v is 1 only for one clock cycle (when count resets to 0). Otherwise, it remains 0.
- This type of horizontal counter is typically used in video processing or VGA signal generation.



Concept check:

Assuming that a time unit, in above mentioned testbench, is defined in nano seconds; what is the frequency at which the designed `h_counter` module would operate?

125 kHz.

Task b.

Code: Design module `v_counter` and its testbench

Provide appropriately commented code for designed module and the testbench, the, code should contain meaningful variable naming.

Modify the testbench provided in **Error! Reference source not found.**(b) to verify the functionality of your designed `v_counter` module

**Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.*

**DESIGN MODULE:-**

```
Project Summary x v_counter.v x clk_div_org.v x clk_div_tb_org.v x test_h_counte < > ? □
C:/Users/Administrator/Desktop/project_3/project_3/srcs/sources_1/new/v_counter.v

1  `timescale 1ns / 1ps
2
3
4  module v_counter(
5      input clk,
6      input v_enable,
7      output reg [9:0] v_count
8  );
9  initial begin
10     v_count = 10'b000_000_000_0;
11
12 end
13
14 always @(posedge clk) begin
15     if (v_count == 10'b100_001_110_0) begin
16         v_count <= 10'b000_000_000_0;
17     end
18     else if (v_enable == 1) begin
19         v_count <= v_count + 1'b1;
20     end
21     else
22     begin
23
24     end
25 end
26
27 endmodule
```


**TEST BENCH:-**

C:/Users/Administrator/Desktop/project_3/project_3/project_3.srscs/sim_1/new/test_v_vounter.v



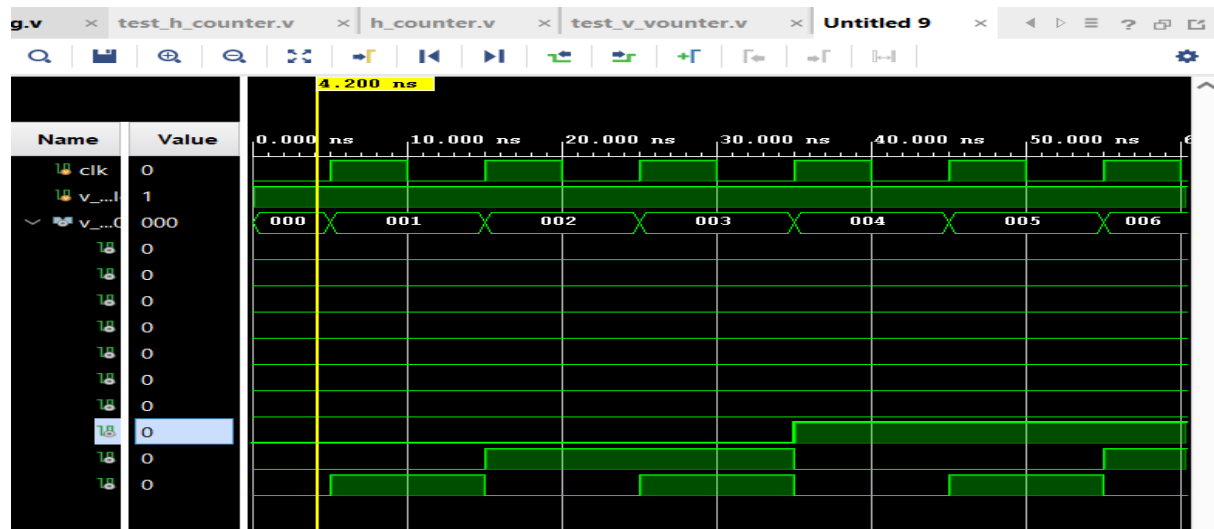
```
1  `timescale 1ns / 1ps
2
3  module test_v_counter();
4      reg clk;
5      reg v_enable;
6      wire [9:0] v_count;
7
8      v_counter c1 (.clk(clk), .v_enable(v_enable), .v_count(v_count));
9
10     // Generate clock with 5 ns period (100 MHz clock)
11     initial clk = 1'b0;
12     always #5 clk = ~clk;
13
14     // Stimulus for the vertical counter (v_enable toggles)
15     initial begin
16         // Initial state: enable the counter
17         v_enable = 1;
18
19         // Change enable signal at different time intervals
20         #100 v_enable = 0;        // Disable for 100 ns
21         #100 v_enable = 1;        // Re-enable for 100 ns
22         #15000 $finish;          // End simulation after 16,000 ns
23     end
24
25     // Optionally, print the v_count value at each clock cycle for debugging
26 endmodule
27
```

**Results (Waveforms)**

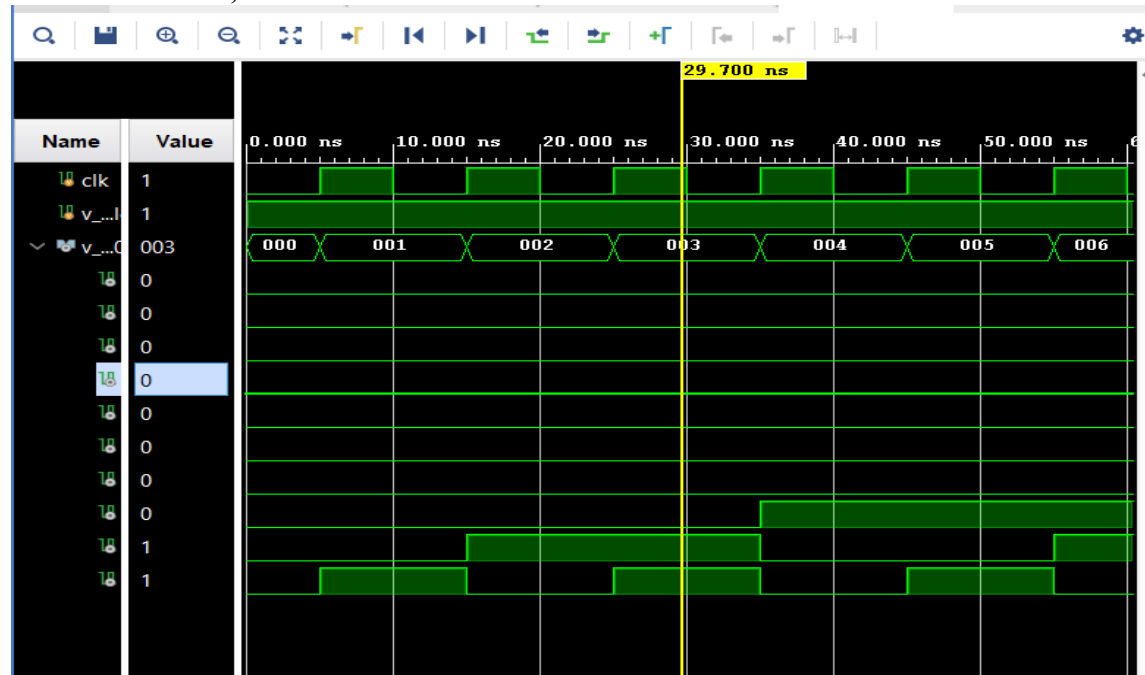
**Add snip of relevant signals' waveforms. Make sure the irrelevant area of the snip is cropped.*

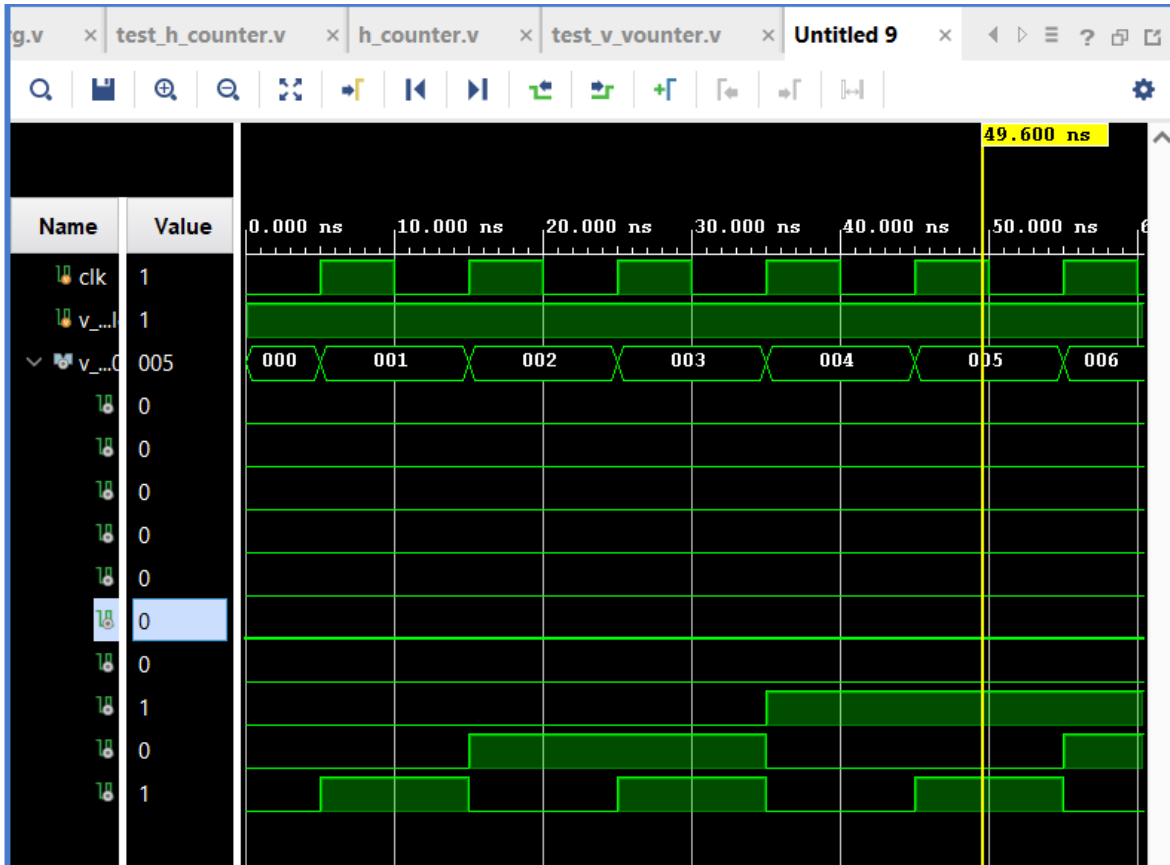
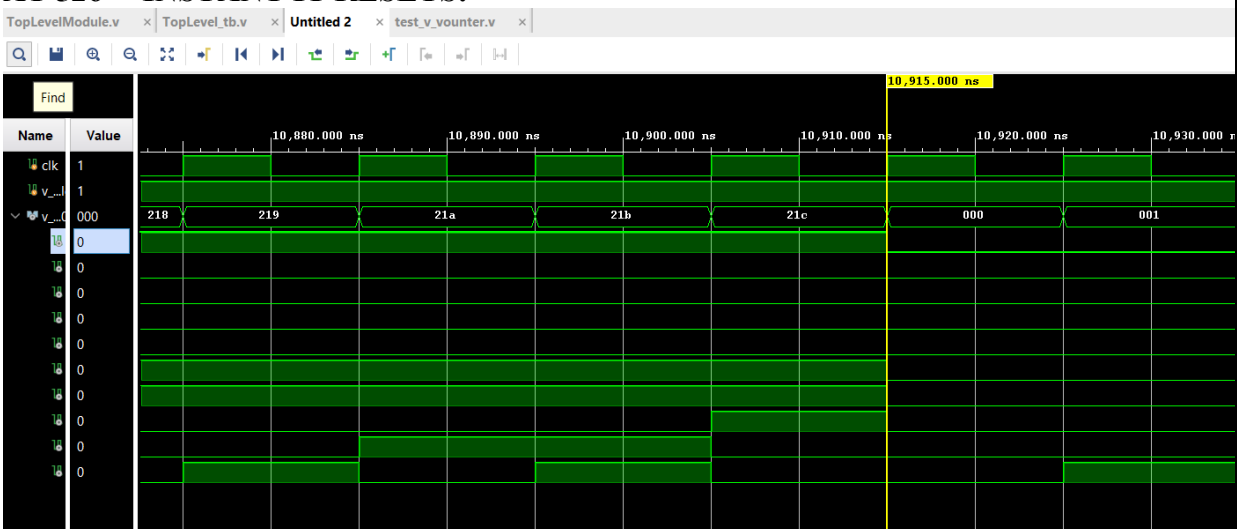
CASE 1

AT INSTANT ZERO, THE COUNT IS ZERO.

**CASE 2**

AT INSTANT 3, THE COUNT IS 3:-



**CASE 3:-
AT 5TH INSTANT THE OUTPUT IS 5****CASE 4:
AT 526TH INSTANT IT RESETS:-**



Comments

**Observation/Comments on the obtained results/working of code.*

- `v_count` is a 10-bit register, allowing values from 0 to 1023. It resets at `10'b100_001_110_0`, which is 526 in decimal. This means the counter counts from 0 to 525, then resets to 0. If `v_enable == 1`, the counter increments normally. If `v_enable == 0`, the counter pauses and does not increment.

**Task c. Clock Divider****Code: Design module & testbench**

Provide appropriately commented code for designed module & its testbench, code should contain meaningful variable naming.

*Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.

DESIGN CODE:-

Project Summary x TopLevelModule.v x TopLevel_tb.v x test_v_vounter.v x v_counter.v

C:/Users/Administrator/Desktop/project_3/project_3/project_3.srscs/sources_1/new/TopLevelModule.v

```
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21
22
23 module TopLevelModule(
24     input clk,
25     output [9:0] h_count,
26     output [9:0] v_count
27 );
28     wire clk_d;
29     wire trig_v;
30     clk_div_org div1 (clk, clk_d);
31     h_counter h1 (h_count, trig_v, clk_d);
32     v_counter v1 (clk_d, trig_v, v_count);
33
34 endmodule
35
```

**TEST BENCH:-**

Project Summary x TopLevelModule.v x TopLevel_tb.v x test_v_vounter.v x v_counter.v x

C:/Users/Administrator/Desktop/project_3/project_3/srcs/sim_1/new/TopLevel_tb.v

Q [Icons]

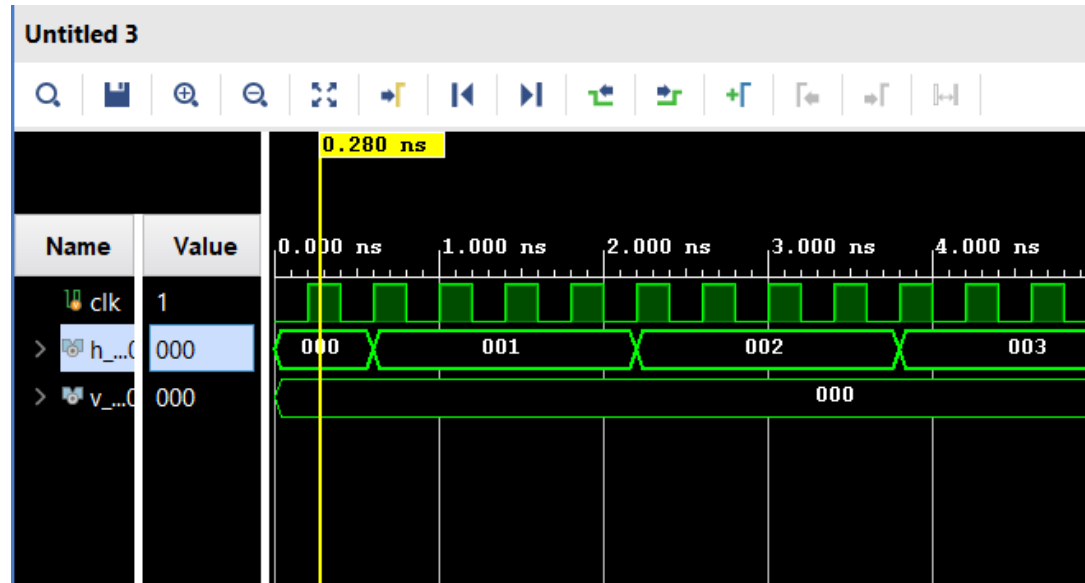
```
15 // Dependencies:
16 //
17 // Revision:
18 // Revision 0.01 - File Created
19 // Additional Comments:
20 //
21 ///////////////////////////////////////////////////////////////////
22
23 module TopLevelModule_tb;
24
25     // Inputs
26     reg clk;
27
28     // Outputs
29     wire [9:0] h_count;
30     wire [9:0] v_count;
31
32     // Instantiate the Unit Under Test (UUT)
33     TopLevelModule uut (
34         .clk(clk),
35         .h_count(h_count),
36         .v_count(v_count)
37     );
38
39     // Generate a clock signal with a period of 10ns (100MHz)
40     initial begin
41         clk = 0;
42         forever #0.2 clk = ~clk;
43     end
44
45     // Stimulus process to test the behavior of the module
46
47 endmodule
48
```



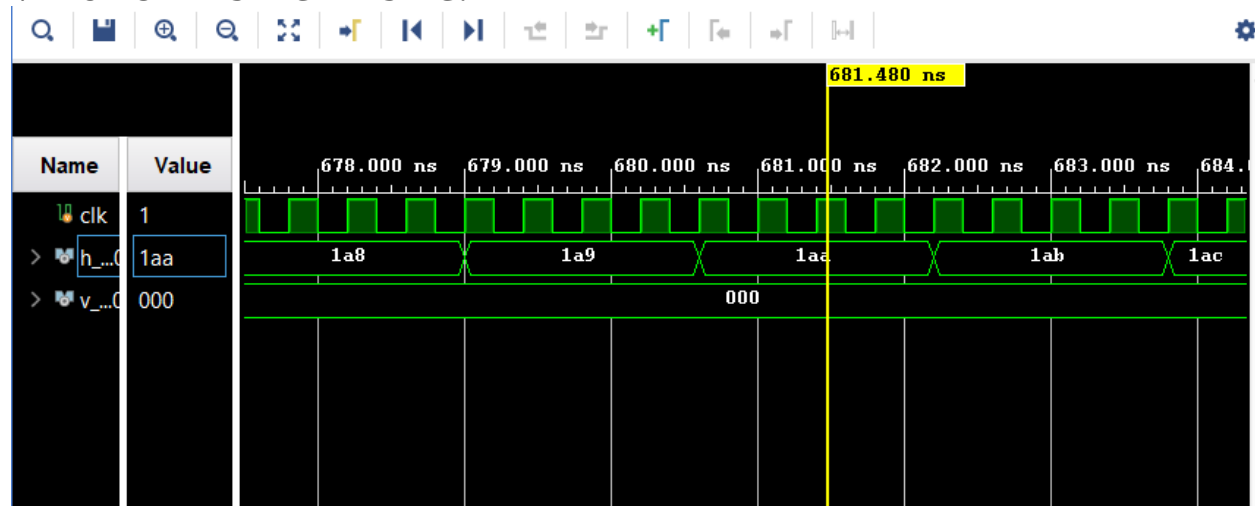
Results (Waveforms)

**Add snip of relevant signals' waveforms. Make sure the irrelevant area of the snip is cropped.*

CASE1 :- THE FIRST VALUE OF H=0 AND V=0

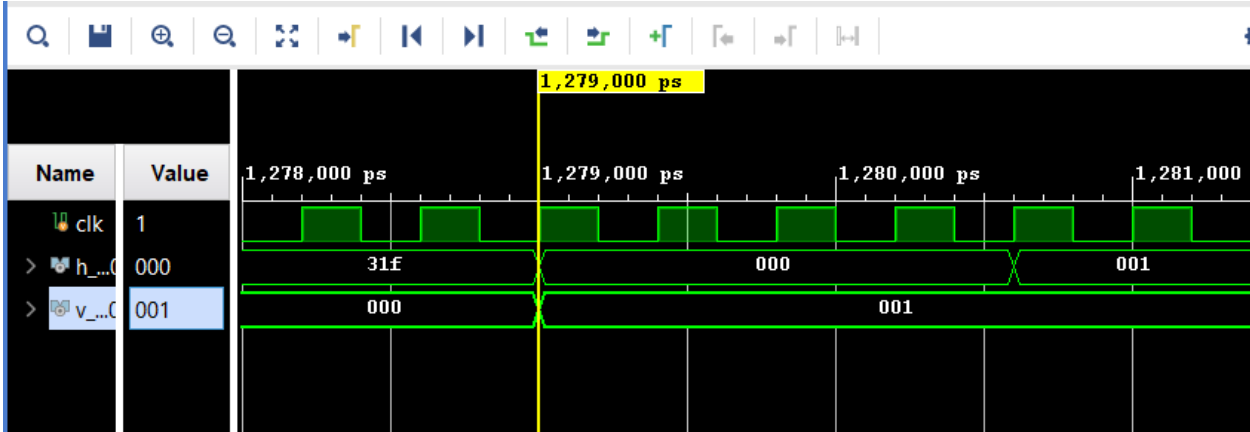


CASE 2:- AT INSTANT BEFORE 799 ie, 681, THE VALUE OF V IS STILL ZERO BUT THE VALUE OF H IS INCREASING.

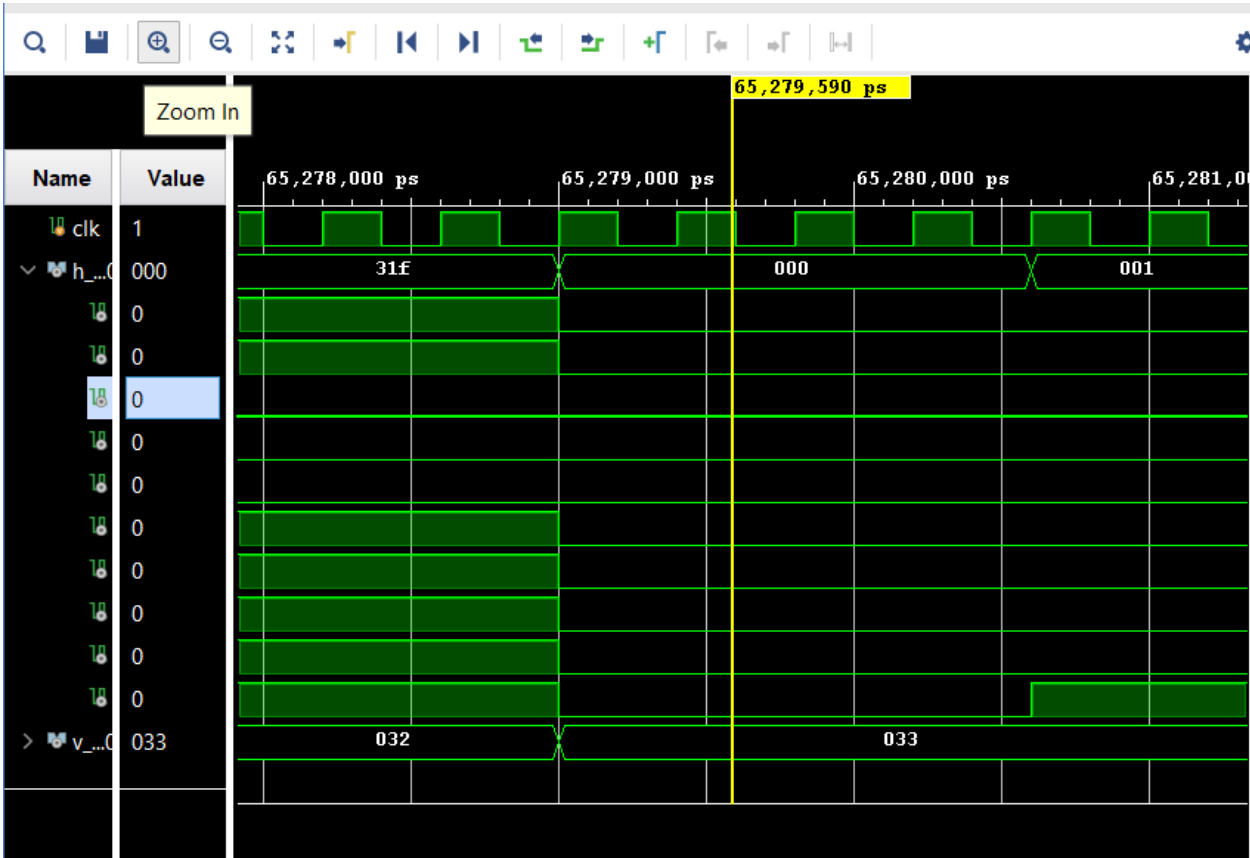




CASE 3:- AFTER THE INSTANT 799, THE VALUE OF V IS NOW 1 AND H RESETS TO ZERO.



CASE 4: AS SOON AS THE H VALUE REACHES ITS MAX, IT RESETS AND THERE IS AN INCREMENT IN V:





Comments

**Observation/Comments on the obtained results/working of code.*

The clock (`clk`) is toggled every #0.2, meaning the clock period is 0.4 ns, which results in a 2.5 GHz frequency. This module combines three submodules: Clock Divider (`clk_div_org`) → Generates a slower clock (`clk_d`). Horizontal Counter (`h_counter`) → Counts pixels per line and triggers the vertical counter. Vertical Counter (`v_counter`) → Counts lines per frame. The clock divider (`clk_div_org`) can be modified to adjust the frequency for different resolutions or refresh rates.

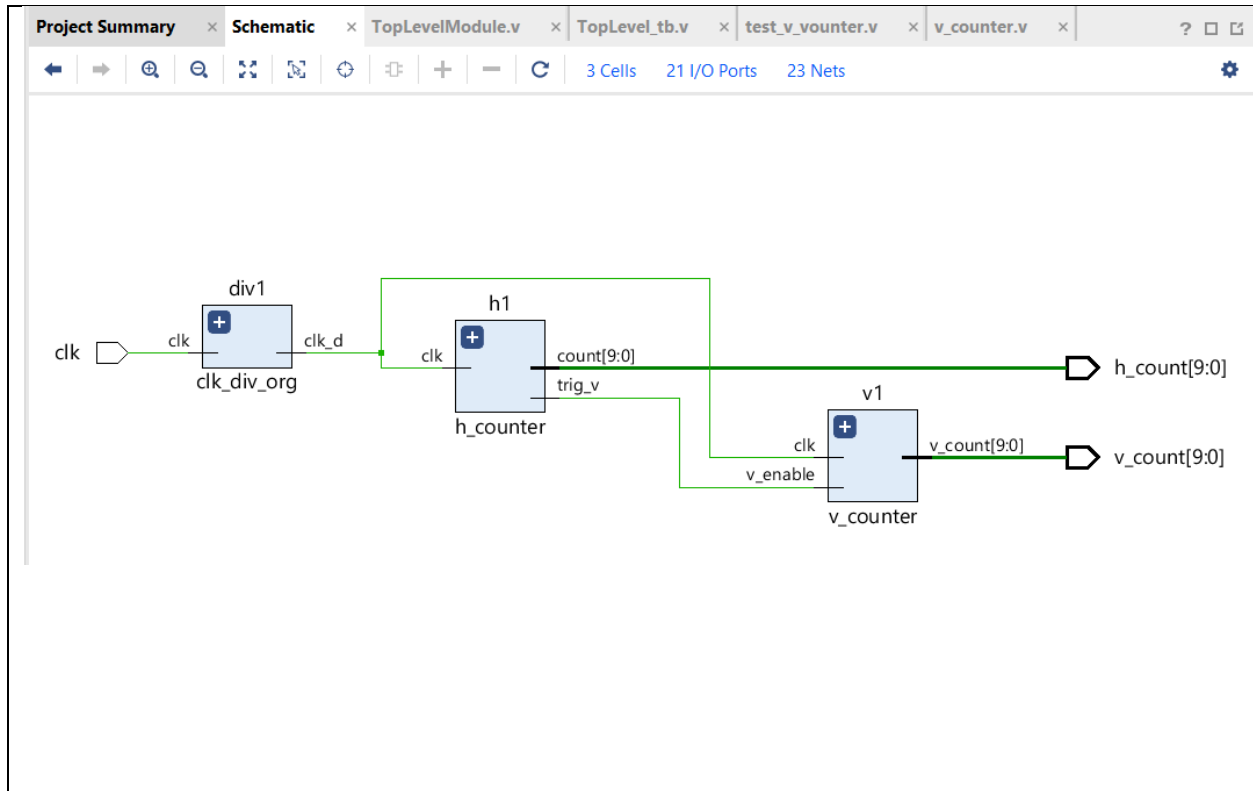
Exercise

Show your Lab results to the RA and get it signed by the RA.

--



RTL Schematic:



Assessment Rubric Lab 10 Worksheet Counter and Clock divider

Name:	Student ID:
-------	-------------

Marks Distribution:

		LR2 Code	LR5 Results
In - Lab	Task a	20	10
	Task b	10	10



	Task c	-	-
Exercise		20	20
Total Marks: 100		/50	/40
CLO Mapping		CLO2	CLO2

Affective Domain Rubric		Points	CLO Mapped
AR 7	Viva	/10	CLO 4

CLO	Total Points	Points Obtained
2	90	
4	10	
Total	100	

For description of different levels of the mapped rubrics, please refer to the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

**Lab Evaluation Rubrics**

#	Assessment Elements	Level 1: Unsatisfactory	Level 2: Developing	Level 3: Good	Level 4: Exemplary
LR2	Program/Code/ Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR7	Viva	Response shows a complete lack of understanding of the assigned / completed task	Response shows shallow understanding of the assigned task	Response shows substantial understanding of the assigned task	Response shows complete understanding of the completed task. The student is able to explain all the related concepts.